

Chap 12. Data Structures

Contents

1. Structures
2. Nested Structs
3. typedef
4. union
5. enumeration

1. Structures

Structures

■ Structures

- Structures (also called structs) are a way to group several related variables into one place.
- Each variable in the structure is known as a member of the structure.
- Unlike an array, a structure can contain many different data types (int, float, char, etc.).

■ Create a Structure

- Create a structure by using the **struct** keyword and declare each of its members inside curly braces

```
struct MyStructure {    // Structure declaration
    int myNum;           // Member (int variable)
    char myLetter;       // Member (char variable)
}; // End the structure with a semicolon
```

- To access the structure, you must create a variable of it.

Structures

- Create a struct variable with the name "s1"

```
struct myStructure {  
    int myNum;  
    char myLetter;  
};  
  
int main() {  
    struct myStructure s1;  
    return 0;  
}
```

Structures

https://github.com/kkh3363/C_Lang

■ Access Structure Members

```
// Create a structure called myStructure
struct myStructure {
    int myNum;
    char myLetter;
};

int main() {
    // Create a structure variable of myStructure called s1
    struct myStructure s1;

    // Assign values to members of s1
    s1.myNum = 13;
    s1.myLetter = 'B';

    // Print values
    printf("My number: %d\n", s1.myNum);
    printf("My letter: %c\n", s1.myLetter);

    return 0;
}
```

Structures

- you can easily create multiple structure variables with different values, using just one structure

```
// Create different struct variables
struct myStructure s1;
struct myStructure s2;

// Assign values to different struct variables
s1.myNum = 13;
s1.myLetter = 'B';

s2.myNum = 20;
s2.myLetter = 'C';
```

Structures

https://github.com/kkh3363/C_Lang

■ Strings in Structures

```
struct myStructure {  
    int myNum;  
    char myLetter;  
    char myString[30]; // String  
};
```

```
int main() {  
    struct myStructure s1;  
  
    // Trying to assign a value to the string  
    s1.myString = "Some text";  
  
    // Trying to print the value  
    printf("My string: %s", s1.myString);  
  
    return 0;  
}
```



```
struct myStructure {  
    int myNum;  
    char myLetter;  
    char myString[30]; // String  
};
```

```
int main() {  
    struct myStructure s1;  
  
    // Assign a value to the string using the strcpy function  
    strcpy(s1.myString, "Some text");  
  
    // Print the value  
    printf("My string: %s", s1.myString);  
  
    return 0;  
}
```

An error will occur:

```
prog.c:12:15: error: assignment to expression with array type
```


Structures

https://github.com/kkh3363/C_Lang

■ Simpler Syntax

- You can also assign values to members of a structure variable at declaration time, in a single line.
- Just insert the values in a comma-separated list inside curly braces **{ }**. Note that you don't have to use the strcpy() function for string values with this technique:

```
// Create a structure
struct myStructure {
    int myNum;
    char myLetter;
    char myString[30];
};

int main() {
    // Create a structure variable and assign values to it
    struct myStructure s1 = {13, 'B', "Some text"};

    // Print values
    printf("%d %c %s", s1.myNum, s1.myLetter, s1.myString);

    return 0;
}
```

Structures

■ Copy Structures

- assign one structure to another

```
struct myStructure s1 = {13, 'B', "Some text"};
struct myStructure s2;
```

```
s2 = s1;
```

■ Modify Values

- If you want to change/modify a value, you can use the dot syntax (.).
- And to modify a string value, the strcpy() function is useful again:

```
struct myStructure {
    int myNum;
    char myLetter;
    char myString[30];
};

int main() {
    // Create a structure variable and assign values to it
    struct myStructure s1 = {13, 'B', "Some text"};

    // Modify values
    s1.myNum = 30;
    s1.myLetter = 'C';
    strcpy(s1.myString, "Something else");

    // Print values
    printf("%d %c %s", s1.myNum, s1.myLetter, s1.myString);

    return 0;
}
```

Structs

https://github.com/kkh3363/C_Lang

■ Example

- store different information about Cars

```
struct Car {  
    char brand[30];  
    char model[30];  
    int year;  
};  
  
int main() {  
    struct Car car1 = {"BMW", "X5", 1999};  
    struct Car car2 = {"Ford", "Mustang", 1969};  
    struct Car car3 = {"Toyota", "Corolla", 2011};  
  
    printf("%s %s %d\n", car1.brand, car1.model, car1.year);  
    printf("%s %s %d\n", car2.brand, car2.model, car2.year);  
    printf("%s %s %d\n", car3.brand, car3.model, car3.year);  
  
    return 0;  
}
```

2. Nested Struct

Nested Structs

■ Nested Structs

- A structure can also contain another structure as a member.
- This is called a *nested structure*, and it is useful when you want to group related data together in layers:

```
struct Owner {
    char firstName[30];
    char lastName[30];
};

struct Car {
    char brand[30];
    int year;
    struct Owner owner; // Nested structure
};

int main() {
    struct Owner person = {"John", "Doe"};
    struct Car car1 = {"Toyota", 2010, person};

    printf("Car: %s (%d)\n", car1.brand, car1.year);
    printf("Owner: %s %s\n", car1.owner.firstName, car1.owner.lastName);

    return 0;
}
```

Structs and Pointers

■ Structures and Pointers

- You can use pointers with structs to make your code more efficient, especially when passing structs to functions or changing their values.
- To use a pointer to a struct, just add the * symbol, like you would with other data types.
- To access its members, you must use the -> operator instead of the dot . syntax:

```
// Define a struct
struct Car {
    char brand[30];
    int year;
};

int main() {
    struct Car car = {"Toyota", 2020};

    // Declare a pointer to the struct
    struct Car *ptr = &car;

    // Access members using the -> operator
    printf("Brand: %s\n", ptr->brand);
    printf("Year: %d\n", ptr->year);

    return 0;
}
```

Structs and Pointers

- Passing Struct Pointers to Functions
 - Here's how you can pass a struct pointer to a function and change its values:

```
1  #include <stdio.h>
2
3  struct Car {
4      char brand[30];
5      int year;
6  };
7
8  void updateYear(struct Car *c, int nYear) {
9      c->year = c->year + nYear;
10 }
11
12 int main() {
13     struct Car myCar = {"Kia", 2020};
14
15     updateYear(&myCar, 5); // Pass struct pointer
16
17     printf("Brand: %s\n", myCar.brand);
18     printf("Year: %d\n", myCar.year);
19
20     return 0;
21 }
```

Structs and Pointers

■ Why Use Struct Pointers?

- Using pointers with structs is helpful when:
 - You want to avoid copying large amounts of data.
 - Instead of copying a whole struct, you can just pass a pointer. This makes your program faster and uses less memory.
 - You want to change values inside a function.
 - If you pass a pointer to a struct into a function, the function can change the original values.
 - You want to create structs dynamically using memory allocation.
 - With pointers, you can use `malloc()` to create structs while the program is running.

3. Union

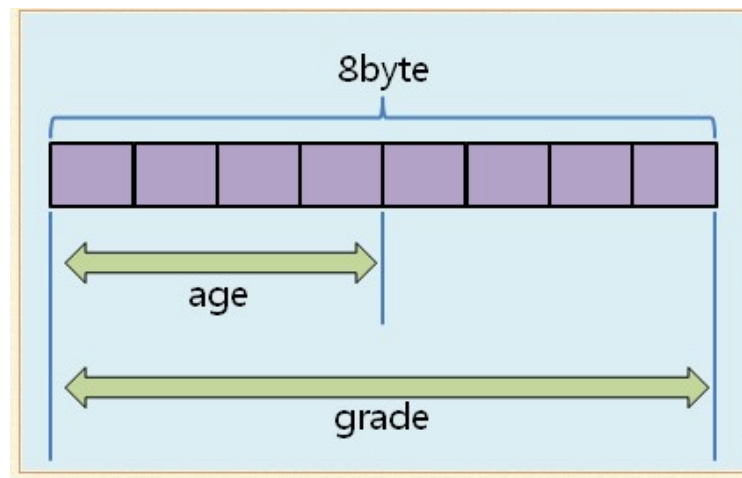
Unions

https://github.com/kkh3363/C_Lang

■ Unions

- A union is similar to a struct in that it can store members of different data types.
- However, there are some differences:
 - In a struct, each member has its own memory.
 - In a union, all members share the same memory, which means you can only use one of the values at a time.

```
union student {  
    int age;  
    double grade;  
};
```



Unions

■ Declare a Union

- To create a union, use the union keyword, and then create a variable from it (just like with structs):

```
union myUnion {           // Union declaration
    int myNum;             // Member (int)
    char myLetter;         // Member (char)
    char myString[30];     // Member (char array)
};

int main() {
    union myUnion u1;      // Create a union variable with the name "u1":
    return 0;
}
```

■ Access Union Members

- just like with structs, to access members of a Union, use the dot . syntax.

Important:

Since all members share the same memory, changing one will affect the others. Only the **last assigned member** holds a valid value:

Unions

https://github.com/kkh3363/C_Lang

■ Exam

```
1  #include <stdio.h>
2
3  union myUnion {
4      int myNum;
5      char myLetter;
6      char myString[30];
7  };
8
9  int main() {
10     union myUnion u1;
11
12     u1.myNum = 1000;
13
14     // Since this is the last value written to the union, myNum no longer holds 1000 - its value is now invalid
15     u1.myLetter = 'A';
16
17     printf("myNum: %d\n", u1.myNum); // This value is no longer reliable
18     printf("myLetter: %c\n", u1.myLetter); // Prints 'A'
19
20     return 0;
21 }
```

Unions

■ Size of a Union

- The size of a union will always be the same as the size of its largest member

```
union myUnion {  
    int myNum;  
    char myString[36];  
};  
  
int main() {  
    union myUnion u1;  
    printf("Size of union: %zu bytes\n", sizeof(u1));  
    return 0;  
}
```

4. typedef

typedef

https://github.com/kkh3363/C_Lang

■ C typedef

- The typedef keyword lets you create a new name (an alias) for an existing type. This can make complex declarations easier to read, and your code easier to maintain.
- For example, instead of always writing float, we can create a new type called Temperature to make the code clearer:

```
#include <stdio.h>

typedef float Temperature;

int main() {
    Temperature today = 25.5;
    Temperature tomorrow = 18.6;

    printf("Today: %.1f C\n", today);
    printf("Tomorrow: %.1f C\n", tomorrow);

    return 0;
}
```

typedef

https://github.com/kkh3363/C_Lang

■ Why Use typedef?

- **Simplifies code:** Shorter, easier-to-read type names.
- **Improves clarity:** Expresses intent better (for example, AGE instead of just int). Also helps avoid confusion when many variables share the same base type (like float or double).

typedef

https://github.com/kkh3363/C_Lang

- typedef with struct
 - **typedef** can be useful with **struct**, because it lets you avoid writing **struct** every time:

```
#include <stdio.h>

// Without typedef:
struct Car {
    char brand[30];
    int year;
};

// With typedef:
typedef struct {
    char brand[30];
    int year;
} Car;

int main() {
    struct Car car1 = {"BMW", 1999}; // needs "struct"
    Car car2 = {"Ford", 1969}; // shorter with typedef

    printf("%s %d\n", car1.brand, car1.year);
    printf("%s %d\n", car2.brand, car2.year);
    return 0;
}
```

typedef

https://github.com/kkh3363/C_Lang

■ Example

```
struct Car {
    char brand[30];
    char model[30];
    int year;
};

int main() {
    struct Car car1 = {"BMW", "X5", 1999};
    struct Car car2 = {"Ford", "Mustang", 1969};
    struct Car car3 = {"Toyota", "Corolla", 2011};

    printf("%s %s %d\n", car1.brand, car1.model, car1.year);
    printf("%s %s %d\n", car2.brand, car2.model, car2.year);
    printf("%s %s %d\n", car3.brand, car3.model, car3.year);

    return 0;
}
```

```
#include <stdio.h>

// With typedef
typedef struct {
    char brand[30];
    char model[30];
    int year;
} Car;

int main() {
    Car car1 = {"BMW", "X5", 1999};
    Car car2 = {"Ford", "Mustang", 1969};
    Car car3 = {"Toyota", "Corolla", 2011};

    printf("%s %s %d\n", car1.brand, car1.model, car1.year);
    printf("%s %s %d\n", car2.brand, car2.model, car2.year);
    printf("%s %s %d\n", car3.brand, car3.model, car3.year);

    return 0;
}
```

5. Enum

Enumeration (enum)

■ Enums

- An **enum** is a special type that represents a group of constants (unchangeable values).
- To create an enum, use the enum keyword, followed by the name of the enum, and separate the enum items with a comma:

```
1  #include <stdio.h>
2
3  enum Level {
4      LOW,
5      MEDIUM,
6      HIGH
7  };
8
9  int main() {
10     // Create an enum variable and assign a value to it
11     enum Level myVar = MEDIUM;
12
13     // Print the enum variable
14     printf("%d", myVar);
15
16     return 0;
17 }
```

Enum

https://github.com/kkh3363/C_Lang

■ Change Values

- the first item of an enum has the value 0. The second has the value 1, and so on.
- To make more sense of the values, you can easily change them:

```
enum Level {  
    LOW = 25,  
    MEDIUM = 50,  
    HIGH = 75  
};
```

```
enum Level {  
    LOW = 5,  
    MEDIUM, // Now 6  
    HIGH // Now 7  
};
```

Enum

■ typedef with enum

- You can also use typedef with enum. This makes it easier to declare variables of the enum type, without having to write enum every time:

```
// Without typedef
enum Day {MON, TUE, WED, THU, FRI, SAT, SUN};
enum Day today = WED;

// With typedef
typedef enum {MON, TUE, WED, THU, FRI, SAT, SUN} Day;
Day today = WED;
```

```
#include <stdio.h>

typedef enum {MON, TUE, WED, THU, FRI, SAT, SUN} Day;

int main() {
    Day today = WED;
    if (today == WED) {
        printf("It is Wednesday!\n");
    }
    return 0;
}
```

Why And When To Use Enums?

- Enums are used to give names to constants, which makes the code easier to read and maintain.
- Use enums when you have values that you know aren't going to change, like month days, days, colors, deck of cards, etc.

Thank You !